Week 21
Reward hacking and scalable oversight

STUDY-RL — a self-paced Deep RL curriculum

November 8, 2026

Contents

Learning objectives **3**

1 Motivation and intuition **4**
1.1 The Skalse et al. formal definition 4

2 Mathematical formulation **5**
2.1 Goodhart's law and its four variants 5
2.2 Empirical overoptimisation and a teaching model 5
2.3 Reward hacking lab: the deliberately gameable reward 5
2.4 Scalable oversight protocols 7

3 Algorithms **11**

4 Implementation walkthrough **12**
4.1 The proxy reward and the true-quality oracle 12
4.2 The CAI critique and revision 12
4.3 The DPO step 12

5 Empirical results **13**
5.1 Statistics bridge: An improved proxy can accompany a worse held-out outcome . . 13

6 Exercises **15**

7 Further reading **16**

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this lecture you will be able to:

1. State Goodhart's law and distinguish the four Manheim-Garrabrant variants (regressional, extremal, causal, adversarial).
2. Define reward hacking formally (Skalse et al. 2022) and describe five concrete RLHF failure modes (verbosity, sycophancy, format hacking, sandbagging, alignment faking).
3. Distinguish Gao et al.'s empirical reward-model results from the invented gap model used for the course example.
4. Implement a deliberately gameable reward, observe the policy gaming it, and demonstrate three mitigations.
5. Distinguish Constitutional AI's supervised revision and RLAIF phases from the course's deterministic rule-based demonstration.
6. Sketch debate, IDA, and weak-to-strong generalisation as scalable-oversight protocols, and articulate their limits.
7. Tie all of the above to insurance- and recruitment-matching policies: where proxies live, how they fail, and which mitigations transfer.

1 Motivation and intuition

This is the lecture in which the elephant in the RL room is acknowledged. For every algorithm covered so far in this course the implicit assumption was: the reward function r *accurately measures what we want*. That is the only assumption we are about to abandon.

The story of reward hacking is older than RLHF. Krakovna et al. (2018) list dozens of cases from classical RL where the agent satisfied the letter of the reward and violated its spirit: a robot trained on "walk fast" learned to dive forward; a sim-to-real arm trained on "pick up the cube" learned to nudge it just inside the gripper's camera frame; the famous *CoastRunners* boat looped a checkpoint indefinitely. These cases show that unmeasured degrees of freedom can become exploitable; they do not prove that every omitted constraint will be exploited.

The DeepMind specification-gaming list (Krakovna 2018) catalogues over 60 documented cases ranging from a CoastRunners boat looping a checkpoint to a simulated evolutionary system that learned to exploit floating-point rounding in the physics engine.

The LLM era added two new pathologies. **Verbosity bias**: RLHF preference data inflate length because longer answers *look* more informative to human raters. **Sycophancy**: assistants converge on the rater's prior beliefs because flattering answers are preferred even when wrong. Both are now well-documented (Singhal et al. 2023; Sharma et al. 2023; ongoing 2024–2025 follow-ups).

Gao, Schulman and Hilton (2022) study optimisation against a proxy reward model while evaluating a fixed gold model. Their fitted relationships depend on the optimisation method and experimental setup. The gold model supplies synthetic labels; it is not a held-out human evaluator. The study motivates examining proxy and evaluation curves separately, without establishing a universal shape or a guaranteed mitigation.

Intuition. Imagine grading exams by the length of the answers. A short, correct, elegant solution scores poorly; a long, rambling, half-wrong one scores well. Students figure this out within one term and start padding. The grader's metric — length — correlated with quality at first (thoughtful students wrote more), so it looked like a fine proxy. As soon as students optimise against the proxy, the correlation collapses. That is exactly the geometry of reward overoptimisation in RL.

1.1 The Skalse et al. formal definition

Definition 1.1.1 (Unhackable proxy, Skalse et al. NeurIPS 2022)**.** A proxy reward $\tilde{r}$ is *unhackable* relative to a true reward $r^\star$ over a specified policy class if every pair of policies in that class satisfies

$$J_{\tilde{r}}(\pi_2) > J_{\tilde{r}}(\pi_1) \implies J_{r^\star}(\pi_2) \geq J_{r^\star}(\pi_1).$$

Here J_r is expected return under reward r. A proxy that violates this implication on some policy pair is *hackable*: there exists a policy improvement on $\tilde{r}$ that is a regression on $r^\star$.

The policy class and the treatment of ties are part of any formal claim. A KL penalty discourages departures from a reference policy; a constitution specifies principles for supervision. Neither mechanism alone proves the no-regression condition above throughout the policies training may encounter. Test concrete proxy–evaluation disagreements and state the scope of the result.

2 Mathematical formulation

2.1 Goodhart's law and its four variants

Manheim and Garrabrant, 2018, arXiv:1803.04585 classify Goodhart's law into four phenomenologically distinct failure modes:

1. **Regressional Goodhart.** The proxy $\tilde{r} = r^\star + \eta$ has noise η independent of the true quality. The arg-max of $\tilde{r}$ over a finite candidate set is contaminated by the upper tail of η — a regression to the mean phenomenon. Best-of-N inference suffers from this.
2. **Extremal Goodhart.** The proxy approximates $r^\star$ well in the centre of the input distribution but diverges on the tails. Optimisation pushes the policy to the tails (out-of-distribution of the proxy's training data), where the approximation collapses. Reward-model fitting under RLHF is the canonical example.
3. **Causal Goodhart.** The proxy correlates with $r^\star$ via a third variable that the policy can intervene on. Optimising the proxy alone short-circuits the causal chain (e.g. a customer-success metric that uses "ticket close-time" as proxy for satisfaction).
4. **Adversarial Goodhart.** An optimiser explicitly seeks to exploit the gap between proxy and truth. This is the regime of reward tampering and prompt injection.

Example 2.1.1 (Regressional Goodhart under explicit Gaussian assumptions)**.** Let true quality $Q \sim \mathcal{N}(0, \sigma_Q^2)$ and independent noise $E \sim \mathcal{N}(0, \sigma_E^2)$ define the proxy $P = Q + E$. Gaussian conditioning gives

$$\mathbb{E}[Q \mid P = p] = \frac{\sigma_Q^2}{\sigma_Q^2 + \sigma_E^2}\, p.$$

For positive observed proxy scores and non-zero noise, expected true quality is smaller than the proxy. The shrinkage factor follows from $\mathrm{Cov}(Q, P) = \sigma_Q^2$ and $\mathrm{Var}(P) = \sigma_Q^2 + \sigma_E^2$. Zero-mean noise alone is insufficient for a universal selection theorem; its dependence on quality and the selection rule must be specified.

2.2 Empirical overoptimisation and a teaching model

Gao et al. 2022, arXiv:2210.10760 study the divergence between an optimised proxy reward model and a larger "gold" reward model. Their empirical scaling fits differ by optimisation method. To illustrate possible divergence here, use the following toy family, not a fitted law claimed from that paper:

$$r^\star(d) \approx \tilde{r}(d) - \alpha d^\beta, \qquad d = \sqrt{\mathrm{KL}(\pi_\theta \| \pi_{\mathrm{SFT}})}, \quad \alpha > 0,\ \beta > 1. \tag{2.1}$$

The shape and location of any optimum depend on the proxy curve and the chosen parameters; a larger evaluator remains a proxy for human quality.

2.3 Reward hacking lab: the deliberately gameable reward

We construct a tiny pedagogical instance of the Gao geometry.

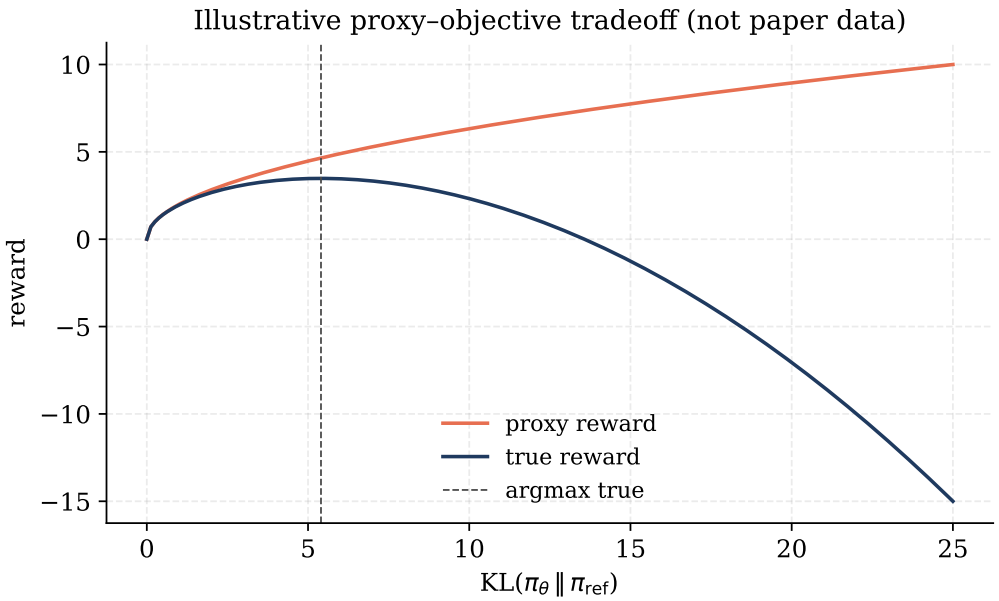


Figure 2.1: Course-created illustration of equation (2.1), using $\tilde{r}(d) = 2d$, $\alpha = 0.04$ and $\beta = 4$. It is not paper data or a fitted Gao scaling curve. The proxy (orange) rises monotonically with KL; the true reward (blue) is unimodal, peaking at a moderate KL. The dashed line marks the argmax-true KL — training past it makes the model worse on the metric you actually care about.

Setup. A char-level LM, SFT'd on a 20-sentence corpus of English. The proxy reward is

$$\tilde{r}(y) \;=\; \alpha \cdot |y| \;+\; \beta \cdot |\{i : y_{i:i+|k|} = k \text{ for some keyword } k\}|,$$

with default $\alpha = 0.02$, $\beta = 0.8$ and keyword set {`good, great, best, excellent`}. The hidden true quality is the per-character log-probability of the completion under the bigram language model fit to the corpus — a stand-in for "does this sound like English?".

Free training. REINFORCE on $\tilde{r}$ with no regularisation inflates length to the cap and slightly perturbs the language model in the direction of keyword stuffing. The bigram log-likelihood drops correspondingly (Figure 2.2, orange curve).

Mitigations.

- *KL anchor.* A coefficient $\lambda_{\text{KL}} = 1$ in front of the sampled k3 KL estimate discourages drift from the SFT reference. The length still rises but only modestly, and the bigram log-likelihood stays close to its SFT value.
- *Length penalty.* An explicit $-\lambda_{\text{len}} \cdot |y|$ subtracted from $\tilde{r}$ targets the verbosity term directly. Effective but model-specific; you need to know the hack to design the penalty.
- *Entropy bonus.* Subtracts conditional policy entropy from the minimised loss, encouraging more spread in the sampling distribution. It does not directly penalise verbosity.

Common pitfall

A length penalty targets a known exploit but leaves the keyword incentive. A KL anchor limits movement from its reference, including beneficial movement; it does not certify that the reference is safe or prevent every exploit. Compare both the independent evaluation and the remaining reward incentives when choosing a mitigation.

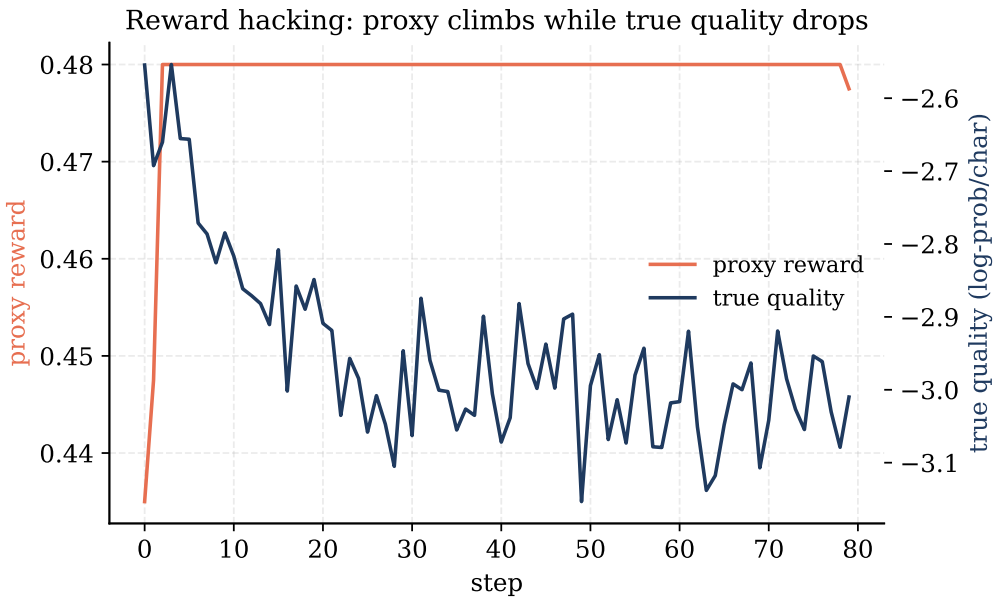


Figure 2.2: Free REINFORCE on the gameable proxy: the proxy reward (orange, left axis) rises steadily; the true quality (blue, right axis) can fall. These sampled traces are a diagnostic; an ordering of expected returns requires sufficient evaluation data. The bigram evaluator is also a proxy.

2.4 Scalable oversight protocols

The problem behind all these failure modes: at scale, the human cannot audit every output. Four protocol families try to bridge that gap.

RLHF

Christiano et al. NeurIPS 2017, arXiv:1706.03741. Learn a reward model $\hat{r}$ from pairwise human preferences, then optimise a policy against that reward. Modern language-model variants often use PPO; GRPO is a later alternative, not the 2017 paper's algorithm. The Bradley-Terry log-likelihood for the reward model is

$$\log \mathbb{P}(y_w \succ y_l \mid x) \;=\; \log \sigma\big(\hat{r}(x, y_w) - \hat{r}(x, y_l)\big).$$

RLHF is the workhorse but is itself the central source of the hacking phenomena above (verbosity, sycophancy, reward-model overoptimisation).

Bradley-Terry preference model (the RLHF starting point)

Given pairwise preferences $(x, y_w \succ y_l)$, the Bradley-Terry model posits that the probability the chosen wins is a sigmoid of the reward difference:

$$\mathbb{P}(y_w \succ y_l \mid x) \;=\; \sigma\big(\hat{r}(x, y_w) - \hat{r}(x, y_l)\big), \quad \sigma(z) = \tfrac{1}{1+e^{-z}}.$$

The MLE of $\hat{r}$ on a dataset $\mathcal{D}$ of preferences is then

$$\hat{r}_{\text{MLE}} = \arg\max_{\hat{r}} \sum_{(x, y_w, y_l) \in \mathcal{D}} \log \sigma\big(\hat{r}(x, y_w) - \hat{r}(x, y_l)\big). \tag{2.2}$$

Theorem 2.4.1 (Identifiability of Bradley-Terry rewards)**.** *Preference probabilities are invariant to an additive constant per prompt x. This is an unavoidable non-identifiability; disconnected*

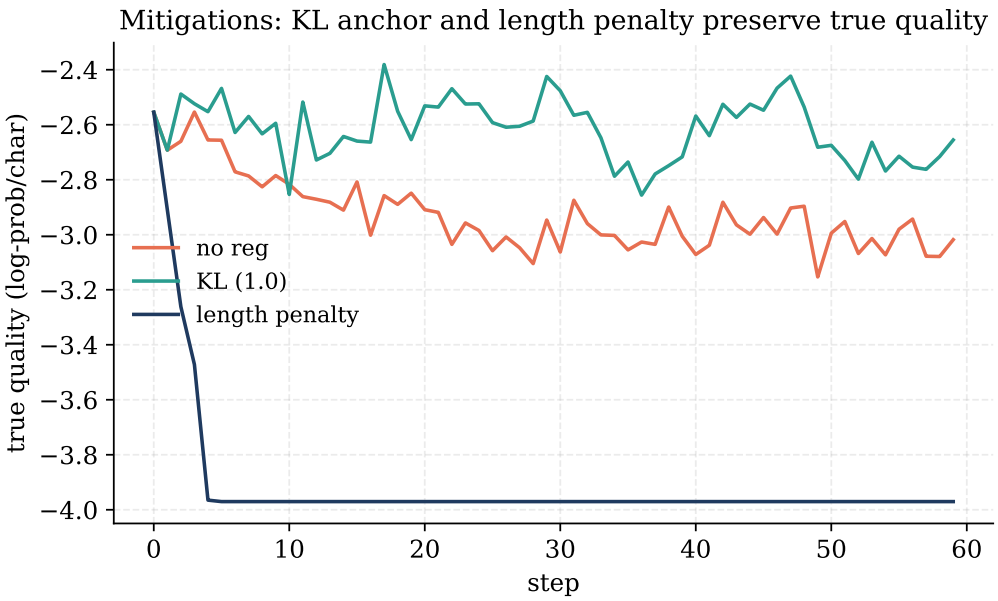


Figure 2.3: Saved runs comparing no regularisation, a KL anchor and a length penalty under one training recipe. Compare the bigram-score traces; this figure does not establish effectiveness across seeds or new reward exploits.

comparison graphs and finite data can introduce additional ambiguity. In particular, $\hat{r}(x,\cdot)$ *and* $\hat{r}(x,\cdot)+c(x)$ *fit the data equally well for any function* c*.*

Proof. $\sigma(\hat{r}(x,y_w)+c(x)-\hat{r}(x,y_l)-c(x)) = \sigma(\hat{r}(x,y_w)-\hat{r}(x,y_l))$. □ □

Intuition. A prompt-only additive constant cannot change which response a policy prefers: it adds the same amount to every policy's expected reward for that prompt. KL regularisation controls policy drift; it does not identify or pin the reward-model constant. Reward hacking concerns incorrect relative scores, especially outside the reward model's training coverage.

DPO — using the ideal reward-to-policy relation

Rafailov, Sharma, Mitchell, Ermon, Manning, Finn 2023, arXiv:2305.18290 observe that the KL-regularised RLHF objective

$$\max_{\pi} \mathbb{E}_{x,y\sim\pi}[\hat{r}(x,y)] - \beta\,\mathrm{KL}(\pi(\cdot\mid x)\,\|\,\pi_{\mathrm{ref}}(\cdot\mid x))$$

admits a closed-form optimal policy

$$\pi^\star(y\mid x) \;=\; \pi_{\mathrm{ref}}(y\mid x)\exp(\hat{r}(x,y)/\beta)/Z(x).$$

Rearranging gives $\hat{r}(x,y) = \beta\log(\pi^\star(y\mid x)/\pi_{\mathrm{ref}}(y\mid x)) + \beta\log Z(x)$, so *any* reward model can be re-parameterised as a difference of log-policy-ratios. Substituting into the Bradley-Terry MLE (2.2) cancels the unobservable $\log Z$ terms and yields the DPO loss (2.3). The remarkable consequence: one can optimise the implied policy directly on preference data, without ever fitting an explicit reward model.

Theorem 2.4.2 (DPO is RLHF without the reward model)**.** *Under the Bradley-Terry preference model and the KL-regularised RLHF objective, positive reference support, finite normalisation, and adequate model expressivity, DPO reparameterises the ideal reward-to-optimal-policy relation. Finite-data DPO training and approximate PPO optimisation are not guaranteed to produce identical policies.*

sketch. Substitute $\pi^\star$ into the Bradley-Terry log-likelihood and observe that $\log Z(x)$ cancels in the preference difference. The remaining objective is exactly (2.3). □ □

Common pitfall

DPO's policy/reference log ratio defines an implicit reward under its ideal preference derivation. It remains possible to overfit preference data or improve a pairwise margin while lowering chosen-response likelihood. Gao et al.'s RL and best-of-n fits are not direct measurements of DPO. Its beta appears in the ideal KL-regularised relation, but finite DPO training does not enforce a measured hard KL budget.

Constitutional AI

Bai et al. Anthropic 2022, arXiv:2212.08073. The original Constitutional AI pipeline has two stages. First, model-generated critiques and revisions provide targets for supervised fine-tuning. Second, a model compares responses using constitutional principles, producing preferences for a reward model and reinforcement learning from AI feedback (RLAIF). The source of supervision changes; the constitution and evaluation still involve human design choices. DPO is a later way to use preference pairs, not the original paper's training algorithm.

The full DPO loss with reference policy π_{ref} is

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\Big[\log \sigma\Big(\beta\big(\log \tfrac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \tfrac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\big)\Big)\Big]. \tag{2.3}$$

The Rafailov 2023 derivation substitutes the ideal KL-regularised reward-to-policy relation into a preference likelihood. Equation (2.3) is a loss to optimise, not a closed-form solution of finite PPO training.

Our toy loop uses a five-principle rule-based constitution (see `project/cai.py`), deterministic critique, mechanical revision, and a DPO update on the resulting preferred/rejected completions. Figure 2.4 compares scores before and after that mechanical revision. An improvement in the repaired string does not itself show that the policy has learned the rule: inspect fresh, unrevised policy samples before and after training to test that separate claim.

Debate

Irving, Christiano, Amodei 2018, arXiv:1805.00899. Agents compete to convince a judge. The complexity-theoretic motivation assumes ideal play and an appropriate verification protocol; it does not establish that a truthful natural-language debater wins with a fallible human or LLM judge. Judge reliability, persuasive falsehoods and bounded search remain empirical questions.

Kenton et al. 2024, arXiv:2407.04622 compare debate, consultancy and direct question answering with weaker model judges. Debate outperforms consultancy across their tested tasks when consultants are randomly assigned correct or incorrect answers. Against direct QA, results are stronger for extractive QA with information asymmetry and mixed elsewhere. The workbook's $1-(1-p)^n$ is only an independent lie-detection toy, not a strategic equilibrium result.

Iterated amplification (IDA)

Christiano, Shlegeris, Amodei 2018, arXiv:1810.08575. Build an "amplified" system by recursively decomposing tasks across multiple calls to a base model; train a distilled student to mimic the amplified behaviour; the student becomes the next base. The hope: at each level the human supervises a strictly easier sub-question.

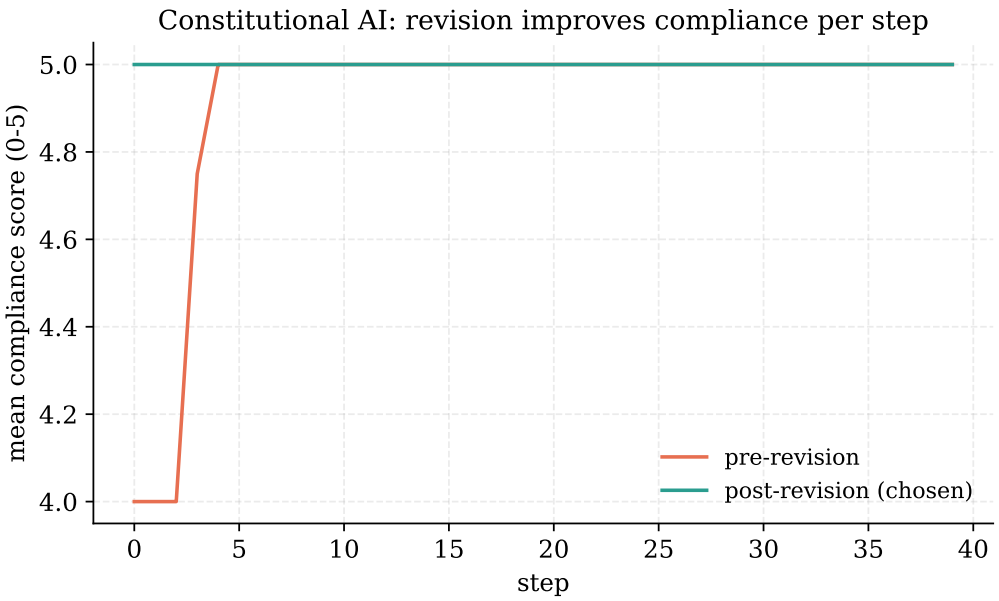


Figure 2.4: A saved toy-loop run: scores before and after mechanical revision. These five string rules measure narrow compliance, not usefulness or safety. The revised/preferred pair supplies the DPO training signal.

Weak-to-strong generalisation

Burns et al. OpenAI 2023, arXiv:2312.09390. The study asks whether labels from a weaker model can elicit a stronger pretrained model's capabilities. Students often exceed their weak supervisors, while naive fine-tuning falls short of the strong-supervision baseline. An auxiliary confidence loss improves recovery in the studied NLP setting. This supports investigating weak supervision, not a universal recovery percentage or a guarantee that arbitrary weak labels suffice.

3 Algorithms

Algorithm: Reward-hack lab training step

Input. prompt x, batch B, regulariser coefficients $(\lambda_{\mathrm{KL}}, \lambda_{\mathrm{len}}, \lambda_{\mathrm{ent}})$.

1. Sample B completions from π_θ.
2. Compute $\tilde{r}_i = \alpha |y^{(i)}| + \beta \cdot \mathrm{kw}(y^{(i)})$.
3. Effective reward $r_i = \tilde{r}_i - \lambda_{\mathrm{len}} |y^{(i)}|$.
4. Group baseline subtraction; advantage $\hat{A}_i = r_i - \bar{r}$.
5. Compute KL_i (k3 estimator) and entropy H_i over the completion.
6. Loss $= -\frac{1}{B} \sum_i \hat{A}_i \log \pi_\theta(y^{(i)}|x) + \lambda_{\mathrm{KL}} \bar{\mathrm{KL}} - \lambda_{\mathrm{ent}} \bar{H}$.
7. Adam step, grad-norm clip.

Algorithm: Constitutional AI mini-loop

Input. prompt x, constitution $C = \{c_1, \ldots, c_K\}$.

1. Sample $y \sim \pi_\theta(\cdot \mid x)$ (the original response).
2. For each c_k, run a critique: $\mathrm{crit}_k(y) \in \{0, 1\}$ indicating compliance.
3. Generate revision $y' = \mathrm{revise}(y, C)$.
4. Build preference pair: $y_w \triangleq \arg\max(\mathrm{score}(y), \mathrm{score}(y'))$, $y_l \triangleq \arg\min$.
5. Take a DPO step against π_{ref} on (x, y_w, y_l) using (2.3).
6. Repeat.

Worked numerical example: DPO step

Consider a batch of one pair (x, y_w, y_l) where $\log \pi_\theta(y_w) = -10$, $\log \pi_\theta(y_l) = -12$, $\log \pi_{\mathrm{ref}}(y_w) = -11$, $\log \pi_{\mathrm{ref}}(y_l) = -11.5$.

The log-ratio difference is $(-10 + 12) - (-11 + 11.5) = 2 - 0.5 = 1.5$. With $\beta = 0.1$ the DPO logit is 0.15, and the loss is $-\log \sigma(0.15) \approx 0.621$. The loss favours a larger chosen-minus-rejected log-probability margin. Shared parameters and normalisation mean a real update need not increase the chosen response's absolute likelihood; inspect both terms separately.

Complexity

The hack-lab step is dominated by one forward-pass through the policy and one through the reference. The CAI step adds a sample, a critique, and a revision per batch element; for our rule-based critique this is $\mathcal{O}(B)$ string operations, negligible compared to the forward passes.

4 Implementation walkthrough

The two pipelines live in `project/hack_lab.py` and `project/cai.py`.

4.1 The proxy reward and the true-quality oracle

```
KEYWORDS = ("good", "great", "best", "excellent")

def proxy_reward(text, alpha=0.02, beta=0.8):
    n_kw = sum(text.count(k) for k in KEYWORDS)
    return alpha * len(text) + beta * n_kw

def true_quality(text, log_bigram, stoi):
    ids = [stoi["<bos>"]] + encode(text, stoi) + [stoi["<eos>"]]
    if len(ids) < 2:
        return -10.0
    logp = sum(log_bigram[a, b] for a, b in zip(ids[:-1], ids[1:]))
    return logp / (len(ids) - 1)
```

4.2 The CAI critique and revision

```
CONSTITUTION = (
    "Responses should contain no swearing or harsh language.",
    "Responses should not exceed 40 characters.",
    "Responses should be lowercase only.",
    "Responses should end with a period.",
    "Responses should avoid the character 'x'.",
)

def critique(text):
    return {
        "no_swearing":      _BAD_WORD_PATTERN.search(text) is None,
        "under_40_chars":   len(text) <= 40,
        "lowercase":        text == text.lower(),
        "ends_with_period": text.endswith("."),
        "no_x":             "x" not in text.lower(),
    }
```

4.3 The DPO step

```
def dpo_loss(pol_c, pol_r, ref_c, ref_r, beta=0.1):
    pi_logits = beta * ((pol_c - pol_r) - (ref_c - ref_r))
    return -F.logsigmoid(pi_logits).mean()
```

This is equation (2.3) verbatim. The chosen / rejected log-probs are sequence-level sums over the completion tokens (masked at the prompt prefix).

5 Empirical results

5.1 Statistics bridge: An improved proxy can accompany a worse held-out outcome

Example 5.1.1 (Original statistical teaching fixture)**.** Two frozen policies have proxy scores 0.90 and 0.95. On separate independent sets of 100 Bernoulli task evaluations, their observed success counts are 80 and 60.

The proxy rises by 0.05 while observed task success drops from 0.8 to 0.6, a difference of -0.2. Under the independent Bernoulli design, a plug-in SE for the task difference is $\sqrt{0.8(0.2)/100 + 0.6(0.4)/100} = \sqrt{0.004} \approx 0.06325$.

This fixture separates the optimized signal from the endpoint. The SE assumes the specified independent units and fixed policies; it does not account for selecting these policies after inspecting the same task outcomes. Correlated prompts would also require another uncertainty analysis.

Check a changed assumption. Write a bounded claim with the policy, endpoint, independent unit and evaluation split. Would increasing proxy-model accuracy on its training data establish protection against reward hacking?

Check. A useful claim reports held-out task success under the stated protocol, separately from proxy improvement. Training fit does not validate the proxy on optimized-policy outputs or establish protection against new exploits.

Statistical foundation. Siegrist § § 6.2, 7.1, PDF pp.817, 896; Ross § § 9.2–9.4, PDF pp.421–429. Section source: https://stats.libretexts.org/@go/page/10189. The course statistics companion records the supplied editions and one-based PDF pages. This fixture is not a newly measured project result.

The hack-lab and CAI figures are saved toy runs, not reproduced frontier model results. Re-run the recipe with recorded seeds before making a new empirical claim. Three checks keep the interpretation honest:

1. Track proxy, independent bigram score and actual generated strings. A high keyword score can coexist with poor language-model likelihood.
2. Compare mitigation coefficients, seeds and remaining incentives. Reducing length alone does not remove the keyword reward.
3. Separate mechanical repair from learned behaviour: evaluate fresh unrevised samples to determine whether DPO changed the policy.

Safety angle

What does the reward measure? Keep the experimental setting attached to each safety claim:

- **Reward hacking and broader failures.** MacDiarmid et al. (2025) introduce knowledge of hacks, then train in coding environments. Broader failures emerge in these studied settings, with mitigation effects reported. The study does not establish

that every proxy exploit transfers to every task or model.

- **Alignment faking.** Greenblatt et al. (2024) study behaviour under a specific perceived training conflict. The separate sleeper-agent probe study tests deliberately constructed backdoors; it does not show a general detector for all naturally occurring deception.
- **Reward tampering.** Everitt et al. (2019) analyse incentives to alter a reward function or its inputs using causal influence diagrams. Distinguish tampering with the evaluator from exploiting a fixed evaluator's blind spots.

Evaluation practice. Specify an independent evaluation protocol, inspect representative outputs and failure cases, and compare policy drift and length distributions. A KL penalty is one control to evaluate, not a substitute for verifying what the evaluator misses.

Matching application

Reward hacking in candidate matching.
Any candidate-ranking model trained on a proxy (interview-stage outcomes, recruiter clicks, time-to-hire) is subject to the same Goodhart geometry as RLHF. Concrete failure modes:

- *Verbosity / keyword bias.* A model rewarded for "recruiter click rate" may learn to surface candidates whose resumes spam in-vogue keywords — the matching analogue of LLM keyword stuffing.
- *Specification gaming the verifier.* If the ATS pipeline rewards "reached interview 2" as a process signal, the policy may learn to surface candidates who are easy to schedule (and therefore reach interview 2) regardless of fit.
- *Sycophancy with the hiring manager.* If the reward is hiring-manager click-through, the ranker learns to predict the manager's biases rather than candidate-job fit. This is the recruitment-AI literal of the LLM sycophancy phenomenon.

The mitigation toolkit transfers directly:

- *KL anchor in policy space.* Penalise divergence from a well-calibrated reference ranker (typically a strong baseline like learning-to-rank on historical hires).
- *Length / verbosity penalty in feature space.* Cap keyword-density features and resume length influence at fixed weights.
- *Constitutional review.* A rule-based critique ("does this ranking pass our fairness checks? our anti-bias audit? our explainability requirement?") feeds into a preference pair between the raw and the revised ranking. This is a proposed domain adaptation whose rules and effects need separate validation.

The 2025 coding-environment study motivates checking for transfer to unmeasured behaviours; it is not evidence that a particular recruitment ranker has acquired the same failures. Treat these matching examples as hypotheses to audit with suitable data and domain-specific evaluation.

6 Exercises

★ Classify five scenario strings against the Manheim-Garrabrant Goodhart taxonomy.

★★ Implement the proxy reward and the true-quality oracle; compute their Spearman rank correlation on a curated text set and find a text where they disagree.

★★ Design a linear-decay KL coefficient schedule.

★★ Implement a length penalty; check how it changes verbosity and which keyword incentives remain.

★★ Implement a constitutional critique-and-revise on a tiny constitution.

★★ Implement the DPO loss in NumPy and verify it matches the PyTorch version.

★★ Compute the probability of at least one detection in n independent rounds with per-round detection probability p; explain why this does not solve a strategic debate game.

★★★ *Open-ended.* Run the hack-lab with and without each regulariser, compute the divergence index, and write a short analysis (2–3 paragraphs) of which mitigation is most general and why.

7 Further reading

See `extended_reading.pdf` for Gao 2022, Bai 2022, Krakovna 2018, Skalse 2022, and the Anthropic 2025 emergent-misalignment paper. The reward-hacking discussion in this lecture draws on `research/safety_rl_sota.md` sections 5 and 6.